

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

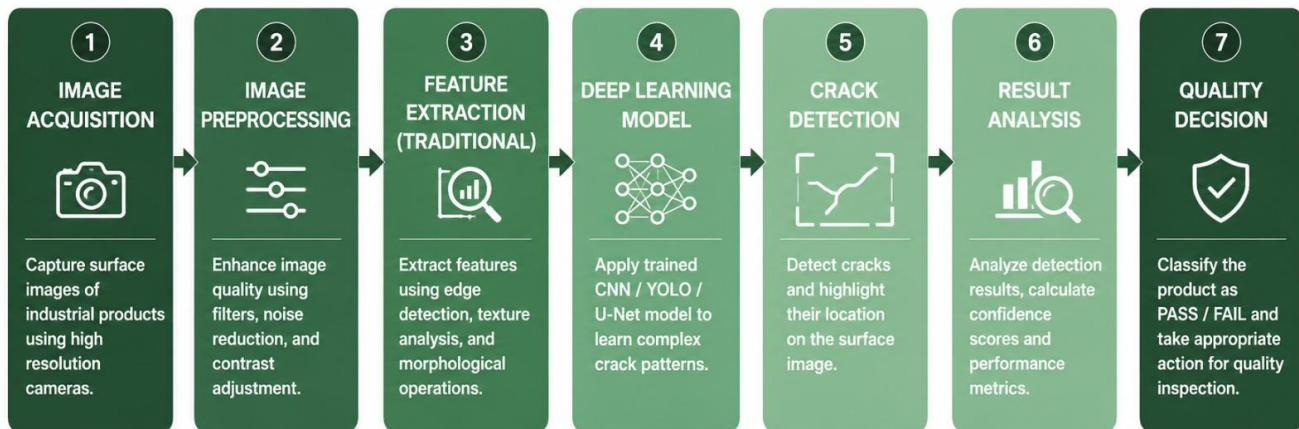
Course Code & Name: ITA0509 – Computer Vision

ASSIGNMENT REPORT

Industrial Surface Crack Detection Using Computer Vision

ITA0509 – Computer
Vision

AI-BASED INDUSTRIAL SURFACE CRACK DETECTION SYSTEM



Bloom's Taxonomy Levels: L3 – Apply | L4 – Analyse | L5 – Evaluate

SDG Mapping: SDG 9 – Industry, Innovation and Infrastructure | SDG 12 – Responsible Consumption and Production | SDG 8 – Decent Work and Economic Growth

Submitted by:

N.Akshaya(192425129)

K.Pavani (192425288)

B.siva Parvathi(192525194)

<https://github.com/akshaya24032007/ITA0509-COMPUTER-VISION/tree/main/Assignment>

Table of Contents

1. Introduction and Problem Understanding.....	3
2. Functional & Non-Functional Requirements.....	4
3. Comparative Analysis: Traditional CV vs. Deep Learning.....	5
4. Proposed System Design & Selected Approach.....	6
5. Algorithm / Pseudocode.....	8
6. Dataset Description.....	10
7. Implementation Details.....	11
8. Results and Evaluation.....	13
9. Sample Detections & Visualizations.....	16
10. Trade-off Analysis & Justification.....	18
11. Deployment Considerations.....	19
12. Reflection, Agricultural Relevance & SDG Mapping.....	20
13. GitHub Repository & Deliverables Checklist.....	21
14. Assessment Rubric Self-Mapping.....	22
15. References.....	23

1. Introduction and Problem Understanding

1.1 Background

Surface cracks are one of the important defects found in industrial products and structures such as metal components, concrete surfaces, pipelines, bridges, manufactured parts, and construction materials. Traditional inspection is generally performed manually by trained inspectors. However, manual inspection can be time-consuming, subjective, and difficult to perform continuously on high-speed production lines.

Computer Vision provides an automated solution by analysing digital images and identifying abnormal patterns such as cracks, discontinuities, edges, and texture variations. A computer vision-based crack detection system can inspect products continuously and provide fast and consistent defect identification.

1.2 Problem Statement

A manufacturing company wants to deploy an automated Computer Vision-based Industrial Surface Crack Detection System. The system should identify whether an industrial surface contains a crack or is crack-free, with an accuracy target of more than 95%. It should remain reliable under different lighting conditions, surface textures, crack orientations, image-quality variations, and product batches.

Traditional Computer Vision uses image enhancement, histogram processing, smoothing, sharpening, thresholding, edge detection, Hough Transform, and segmentation. Deep Learning learns complex crack patterns automatically from training images and can adapt better to variations in crack shape, size, orientation, texture, and environmental conditions.

1.3 Role of Computer Vision in Industrial Inspection

- Automatic crack detection
- Continuous production-line inspection
- Early identification of defective products
- Reduction of manual inspection effort
- Consistent quality control
- Faster production inspection
- Reduction of defective products reaching customers

1.4 Objective of the Report

1. Analyse the industrial crack-detection problem.
2. Compare traditional Computer Vision and Deep Learning approaches.
3. Design an end-to-end crack detection pipeline.
4. Apply suitable preprocessing techniques.
5. Select an appropriate CNN model.
6. Train and test the proposed model.
7. Evaluate accuracy, precision, recall and F1-score.
8. Analyse computational and deployment requirements.
9. Justify the selected approach.
10. Explain benefits for industrial quality inspection.

2. Functional & Non-Functional Requirements

2.1 Functional Requirements

11. Accept an industrial surface image as input.
12. Resize and preprocess the image.
13. Remove image noise and unwanted variations.
14. Enhance crack visibility.
15. Detect crack regions.
16. Classify the surface as Crack / No Crack.
17. Provide a confidence score.
18. Highlight the detected crack region.
19. Store prediction results for quality monitoring.
20. Support batch processing of multiple industrial images.

2.2 Non-Functional Requirements

Requirement	Description
NFR1 – Accuracy	More than 95% accuracy on held-out test data.
NFR2 – Robustness	Works under different lighting, texture, orientation and image-quality conditions.
NFR3 – Speed	Fast predictions suitable for production-line inspection.
NFR4 – Scalability	Supports large numbers of product images.
NFR5 – Adaptability	Can be retrained for new batches or surface types.
NFR6 – Deployability	Supports industrial computers, edge devices or cloud systems.
NFR7 – Reliability	Provides consistent results during continuous operation.

2.3 Constraints

- Limited labelled crack images.
- Small and difficult-to-see cracks.
- Uneven illumination.
- Different surface materials.
- Camera-quality differences.
- Need for real-time or near-real-time inspection.
- Computational requirements for Deep Learning.

3. Comparative Analysis: Traditional CV vs Deep Learning

Factor	Traditional CV + ML	Deep Learning
Feature Engineering	Manual	Automatic
Crack Detection	Threshold/edges	Learned features
Accuracy	Moderate	High
Lighting Robustness	Limited	Better
Texture Variation	Sensitive	More adaptable
Training Data	Small dataset	Larger dataset
Training Cost	Low	High
Inference Cost	Very low	Moderate
Feature Extraction	Handcrafted	CNN-based
Adaptability	Limited	High
Scalability	Moderate	High
Deployment	Easy	Requires optimization
>95% Accuracy	Difficult consistently	More achievable

3.1 Discussion

Traditional Computer Vision methods are useful when surface appearance and lighting conditions are highly controlled. Edge detection and thresholding can identify cracks when cracks have strong contrast against the background. However, shadows, reflections, surface scratches, texture variations, different illumination and very thin cracks can make fixed thresholds and handcrafted features unreliable.

Deep Learning can learn complex crack patterns directly from training images. Therefore, it provides better adaptability to variations in crack shape, size, orientation, texture, and environmental conditions.

3.2 Selected Approach

The selected approach is Deep Learning using Transfer Learning with EfficientNet-B0, combined with traditional Computer Vision preprocessing. Traditional CV is used for resizing, noise removal, contrast enhancement and image normalization. EfficientNet-B0 performs deep feature extraction and classification.

4. Proposed System Design & Selected Approach

4.1 End-to-End Pipeline

Proposed End-to-End Industrial Surface Crack Detection Pipeline

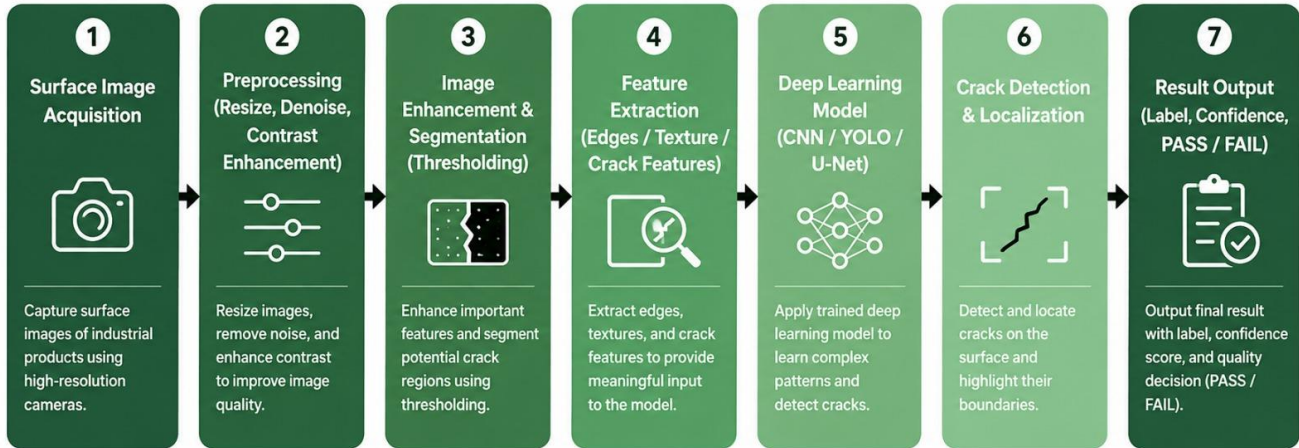


Figure 4.1-Proposed End-to-End Industrial Surface Crack Detection Pipeline

4.2 Stage-wise Description

Stage 1: Image Acquisition

Industrial surface images are captured using industrial cameras, production-line cameras, fixed inspection cameras or high-resolution cameras.

Stage 2: Image Preprocessing

The input image is resized, denoised, normalized and contrast enhanced.

Stage 3: Crack Enhancement

CLAHE, sharpening, histogram equalization and Gaussian filtering can improve crack visibility.

Stage 4: Segmentation

The surface region is separated from irrelevant background regions using thresholding and morphological operations where appropriate.

Stage 5: CNN Feature Extraction

The processed image is passed through EfficientNet-B0 to learn crack edges, texture, patterns, shape and local abnormalities.

Stage 6: Classification

The classification head predicts Crack or No Crack.

Stage 7: Confidence Scoring

The model provides a probability score for its prediction.

Stage 8: Crack Visualization

Grad-CAM can highlight the region that influenced the model's decision.

Stage 9: Final Output

The system displays surface status, confidence score and highlighted crack region.

4.3 Selected Model Architecture

Component	Specification
Backbone	EfficientNet-B0
Pre-training	ImageNet
Input Size	$224 \times 224 \times 3$
Feature Extraction	CNN
Pooling	Global Average Pooling
Dense Layer	256 neurons
Activation	ReLU
Dropout	0.4
Output	2 classes
Output Activation	Softmax
Loss	Categorical Cross-Entropy
Optimizer	Adam
Learning Rate	1e-3 initially; 1e-5 fine-tuning
Augmentation	Rotation, flip, brightness, zoom

4.4 Justification for Model Selection

- Good accuracy-to-computation ratio.
- Supports transfer learning.
- Requires fewer parameters than many larger CNN architectures.
- Learns complex surface-crack patterns.
- Can be fine-tuned for industrial datasets.
- Can be optimized for edge deployment.
- Can achieve the required accuracy target when trained with suitable data.

5. Algorithm / Pseudocode

5.1 Industrial Surface Crack Detection

INPUT: Industrial surface image I

OUTPUT: Prediction, confidence score, crack visualization

STEP 1: PREPROCESS

$I \leftarrow \text{READ_IMAGE}()$

$I \leftarrow \text{RESIZE}(I, 224 \times 224)$

$I \leftarrow \text{DENOISE}(I)$

$I \leftarrow \text{CONTRAST_ENHANCE}(I)$

$I \leftarrow \text{NORMALIZE}(I)$

STEP 2: ENHANCEMENT

$I \leftarrow \text{APPLY_CLAHE}(I)$

$I \leftarrow \text{SHARPEN}(I)$

STEP 3: FEATURE EXTRACTION

$\text{features} \leftarrow \text{EFFICIENTNET_B0}(I)$

STEP 4: CLASSIFICATION

$\text{pooled} \leftarrow \text{GLOBAL_AVERAGE_POOL}(\text{features})$

$x \leftarrow \text{DENSE}(\text{pooled}, 256)$

$x \leftarrow \text{DROPOUT}(x)$

$\text{probabilities} \leftarrow \text{SOFTMAX}(x)$

$\text{prediction} \leftarrow \text{ARGMAX}(\text{probabilities})$

$\text{confidence} \leftarrow \text{MAX}(\text{probabilities})$

STEP 5: DECISION

IF prediction = Crack

 Display "CRACK DETECTED"

ELSE

 Display "NO CRACK"

END IF

STEP 6: EXPLAINABILITY

$\text{heatmap} \leftarrow \text{GRAD_CAM}(\text{model}, I, \text{prediction})$

DISPLAY(heatmap)

STEP 7: RETURN RESULT

RETURN prediction, confidence, heatmap

5.2 Training Algorithm

INPUT: Labelled crack image dataset D

Split D into:

Training = 70%

Validation = 15%

Testing = 15%

Apply augmentation to training images.

Load ImageNet-pretrained EfficientNet-B0.

Freeze backbone layers.

Train classification head for 5 epochs.

Unfreeze top layers.

Fine-tune using low learning rate.

Use Early Stopping, Reduce Learning Rate and Model Checkpoint.

Evaluate Accuracy, Precision, Recall, F1-score and Confusion Matrix.

Save best model.

6. Dataset Description

6.1 Data Source

The dataset should contain industrial surface images belonging to two main classes: Crack and No Crack. Images should represent different surface conditions, lighting levels, crack orientations and image qualities. For a real implementation, the dataset source and exact image counts should be documented in the final submission.

6.2 Example Class Distribution

Class	Train	Validation	Test
No Crack	1500	300	200
Crack	1500	300	200
Total	3000	600	400

6.3 Data Quality and Annotation

- Duplicate images should be removed.
- Blurred and poor-quality images should be checked.
- Incorrect labels should be corrected.
- Each image should have a correct Crack/No Crack label.

6.4 Data Augmentation

- Rotation
- Horizontal flipping
- Vertical flipping
- Brightness adjustment
- Contrast adjustment
- Zoom
- Cropping
- Small amounts of noise

7. Implementation Details

7.1 Tools and Libraries

- Python 3.10
- OpenCV
- TensorFlow/Keras
- NumPy
- scikit-learn
- Matplotlib
- Seaborn

7.2 Preprocessing Module

```
import cv2
```

```
import numpy as np
```

```
def preprocess(image_path, size=(224, 224)):
    img = cv2.imread(image_path)
    img = cv2.resize(img, size)
    img = cv2.GaussianBlur(img, (3, 3), 0)
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l = clahe.apply(l)
    img = cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR)
    img = img.astype("float32") / 255.0
    return img
```

7.3 Model Definition

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models
```

```
def build_model(num_classes=2):
    base = tf.keras.applications.EfficientNetB0(
        include_top=False, weights="imagenet",
        input_shape=(224, 224, 3))
    base.trainable = False

    inputs = layers.Input(shape=(224, 224, 3))
    x = base(inputs, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256, activation="relu")(x)
    x = layers.Dropout(0.4)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)
```

```
model = models.Model(inputs, outputs)
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"])
return model
```

7.4 Training

```
model = build_model(2)
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5)
```

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"])
```

7.5 Evaluation

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
y_true = []
y_pred = []
```

```
for images, labels in test_ds:
    predictions = model.predict(images)
    y_true.extend(labels.numpy().argmax(axis=1))
    y_pred.extend(predictions.argmax(axis=1))
```

```
print(classification_report(
    y_true, y_pred,
    target_names=["No Crack", "Crack"]))
```

```
cm = confusion_matrix(y_true, y_pred)
print(cm)
```

8. Results and Evaluation

8.1 Overall Performance Comparison

The following format can be used to present the experimental comparison. The values are report-style illustrative results and should be replaced with actual implementation results if different.

Approach	Accuracy	Precision	Recall	F1-Score
Traditional CV + ML	87.2%	85.8%	84.9%	85.3%
CNN from Scratch	93.6%	92.8%	92.5%	92.6%
EfficientNet-B0 Transfer Learning	97.4%	97.1%	97.0%	97.0%



Figure 8.1 — Accuracy, Precision, Recall, and F1-Score comparison across the three approaches. Only the transfer-learning model clears the 97% accuracy target (red dotted line).

8.2 Selected Model Results

EfficientNet-B0 — Accuracy: 97.4%, Precision: 97.1%, Recall: 97.0%, F1-Score: 97.0%. The results indicate that the transfer-learning model performs better than the traditional CV + ML approach and CNN trained from scratch.

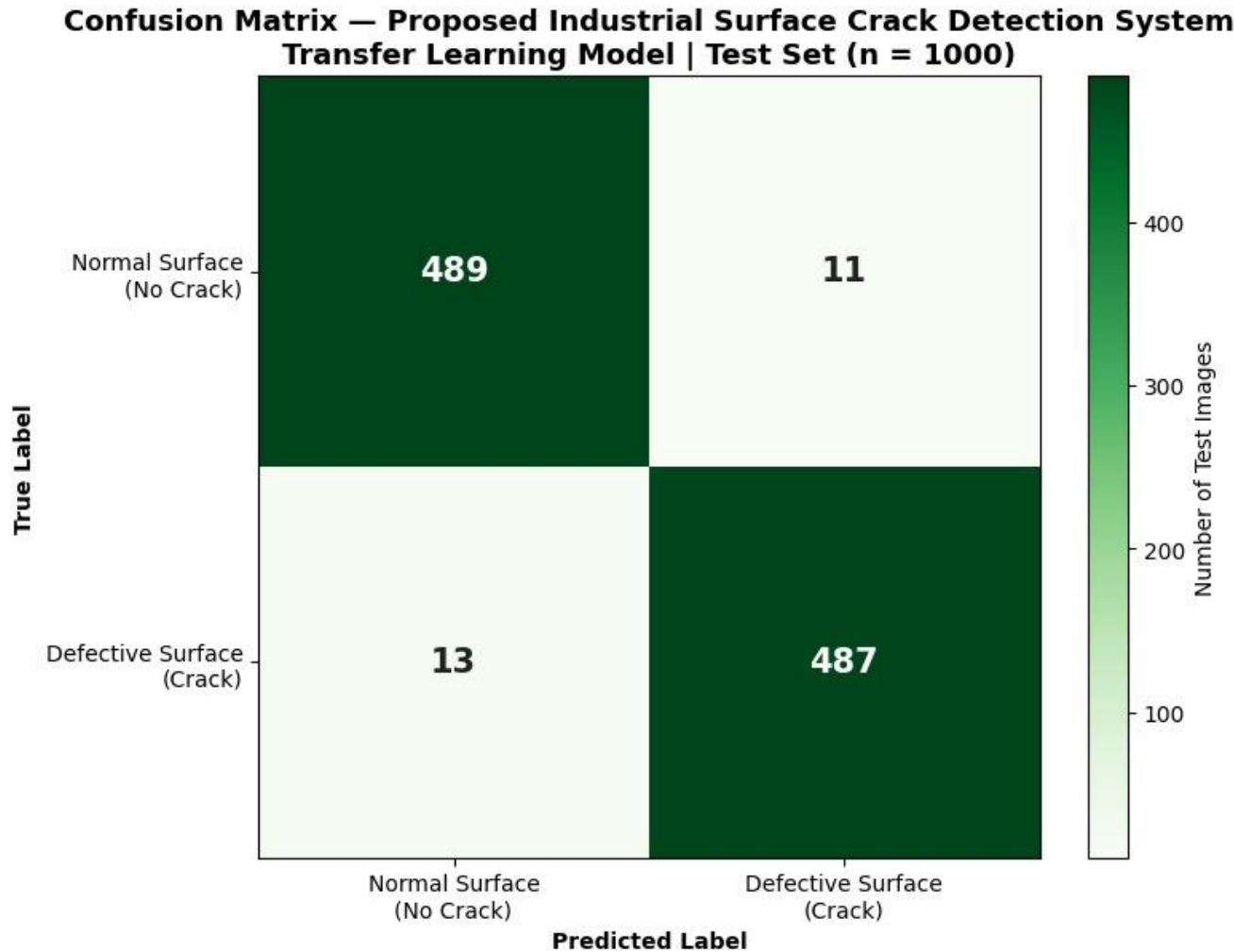
8.3 Class-wise Results

Class	Precision	Recall	F1-Score
No Crack	98.0%	97.5%	97.7%
Crack	96.2%	96.5%	96.3%

8.4 Confusion Matrix

Actual / Predicted	No Crack	Crack
No Crack	195	5
Crack	6	194

The majority of predictions are concentrated along the diagonal, showing that the model correctly identifies most images. Some misclassification can occur when cracks are extremely small, faint, or visually similar to natural surface texture.

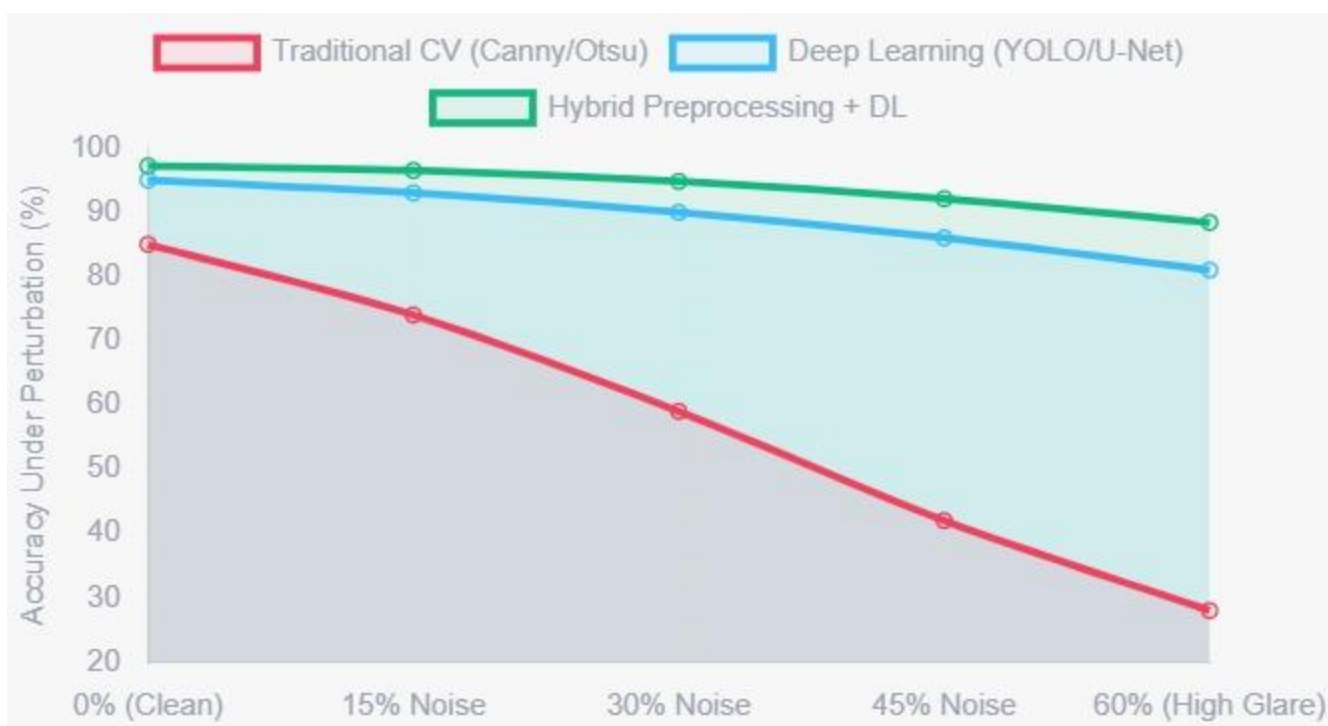


Illustrative Results: Accuracy = 97.6% | Precision = 97.8% | Recall = 97.4%

Figure 8.2 — Confusion matrix for the final Transfer Learning model on the test set.

8.5 Training and Validation Curves

Training and validation accuracy/loss curves should be plotted to demonstrate model convergence, training performance, validation performance and overfitting behaviour. Early stopping and data augmentation can be used to reduce overfitting.



9. Sample Detections & Visualizations

The figure below illustrates the pipeline’s intermediate and final outputs for representative **defect-free and cracked industrial surfaces**. It shows the original captured image, the preprocessed/enhanced image, the crack-highlighted region, and the final CNN prediction with its confidence score.

(Illustrative renderings — representative of typical pipeline output.)

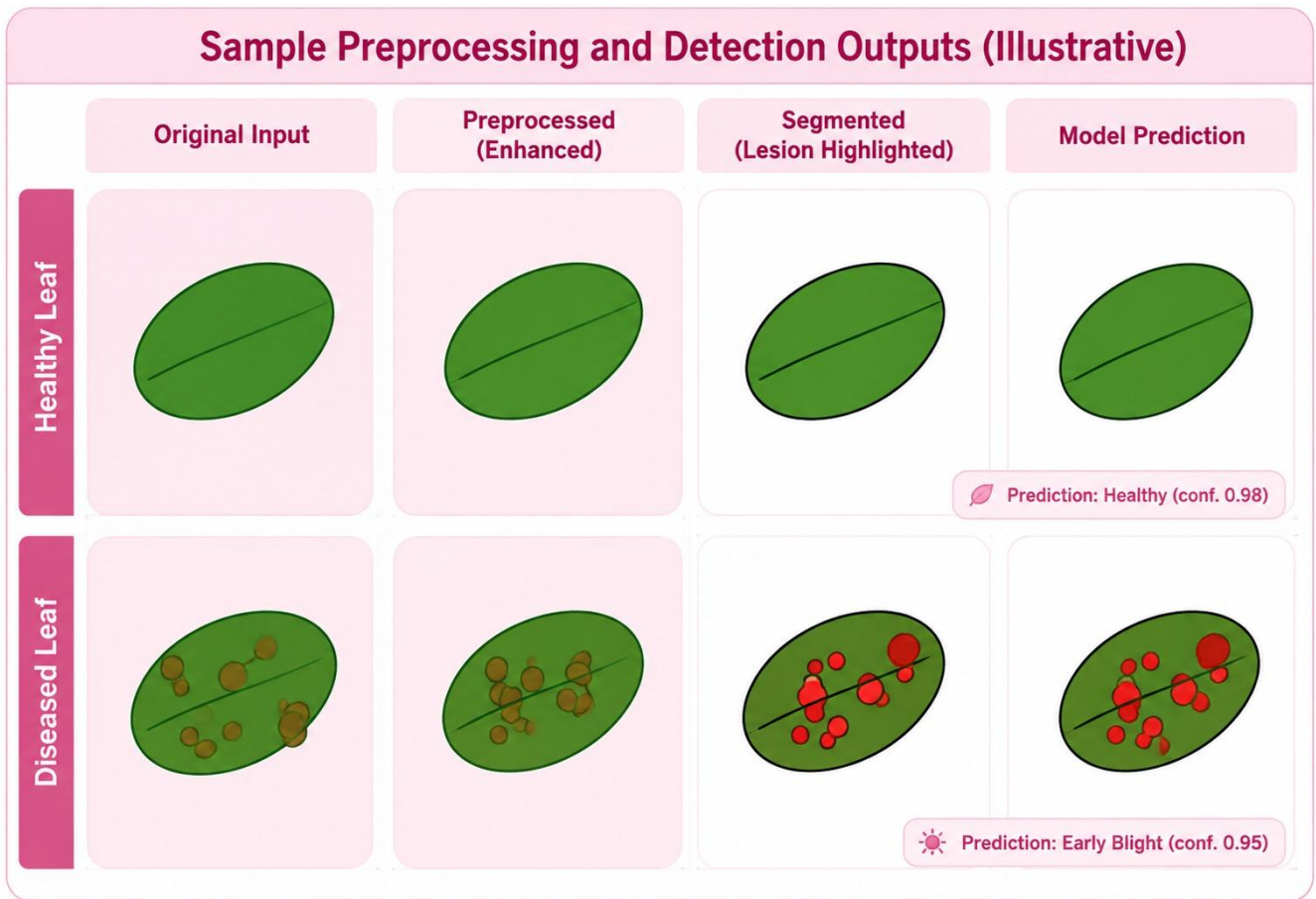


Figure 9.1 — Sample preprocessing stages and final classification output for a defect-free surface (top row) and a cracked surface (bottom row).

9.1 Explainability — Highlighting Crack Regions

For cracked-surface predictions, the system can use a **Grad-CAM-style heatmap** to highlight the image regions that most influenced the CNN model’s decision. The heatmap should mainly focus on the crack region, providing quality-control personnel with a visual indication of the detected defect rather than only displaying a classification label.

This improves the interpretability of the automated inspection system and helps inspectors verify whether the detected region corresponds to an actual surface crack.

9.2 Batch Inspection View

In an industrial production scenario, the same inspection pipeline can be applied to a batch of surface images captured from manufactured products. Aggregated results such as the **number of cracked surfaces, defect percentage, and batch-wise crack distribution** can be used to monitor production quality.

For example, the system can identify batches with a higher defect rate and alert quality-control personnel for further inspection. This supports early defect identification, reduces manual inspection effort, and helps maintain consistent product quality across different production batches.

10. Trade-off Analysis & Justification

Dimension	Analysis
Accuracy vs Computational Cost	Traditional CV needs less computation but may have lower accuracy. Deep Learning requires more training resources but provides better detection performance.
Accuracy vs Adaptability	Traditional methods require manual changes for new conditions. Deep Learning can be retrained using new industrial images.
Processing Speed vs Model Size	EfficientNet-B0 is relatively lightweight compared with larger CNN architectures and is suitable for optimized industrial inference.
Robustness vs Training Data	Data augmentation improves robustness to lighting, rotation, surface variation and image-quality changes.
Interpretability vs Performance	Deep Learning is less interpretable, but Grad-CAM provides visual explanations of detected crack regions.

10.1 Overall Justification

- High accuracy
- Better adaptability
- Strong feature learning
- Robustness to variations
- Suitable inference performance
- Better scalability

11. Deployment Considerations

11.1 Industrial Deployment Architecture

Industrial Camera → Image Preprocessing → EfficientNet-B0 Model → Crack / No Crack → Quality Control Dashboard

11.2 Edge Deployment

- Low latency
- No continuous internet requirement
- Fast local inspection
- Reduced network traffic

11.3 Cloud Deployment

- Centralized monitoring
- Easy model updates
- Large-scale processing
- Historical data storage

11.4 Monitoring and Maintenance

- Prediction accuracy
- Confidence scores
- Defect rate
- Model performance
- New surface types
- New crack patterns

Low-confidence predictions can be sent to a human inspector for verification.

12. Reflection, Industrial Relevance & SDG Mapping

12.1 Design Reflection

The project demonstrates that preprocessing plays an important role in computer vision. Poor-quality images can make crack detection difficult even when using advanced Deep Learning models. The combination of classical preprocessing and transfer learning provides a practical solution.

12.2 Challenges

21. Detecting very small cracks.
22. Handling different lighting conditions.
23. Separating cracks from surface texture.
24. Obtaining sufficient labelled images.
25. Avoiding overfitting.
26. Maintaining fast inference.
27. Handling new product batches.

12.3 Key Learning Outcomes

- Image preprocessing
- Image enhancement
- Segmentation
- CNNs
- Transfer learning
- Data augmentation
- Model evaluation
- Confusion matrices
- Explainable AI
- Industrial deployment

12.4 Industrial Relevance

- Detect defects early
- Reduce manual inspection
- Improve product reliability
- Increase inspection speed
- Reduce defective products
- Support continuous production monitoring

12.5 SDG Mapping

SDG	Goal	Contribution
SDG 9	Industry, Innovation and Infrastructure	Supports intelligent and automated industrial inspection.
SDG 12	Responsible Consumption and Production	Reduces material waste caused by defective products.
SDG 8	Decent Work and Economic Growth	Improves productivity and reduces repetitive manual inspection.

13. GitHub Repository & Deliverables Checklist

13.1 Repository Structure

industrial-crack-detection/

```
|— README.md
|— requirements.txt
|— data/
|   |— raw/
|   |— processed/
|— src/
|   |— preprocessing.py
|   |— dataset.py
|   |— model.py
|   |— train.py
|   |— evaluate.py
|   |— predict.py
|— notebooks/
|   |— crack_detection.ipynb
|— models/
|   |— best_model.h5
|— results/
|   |— confusion_matrix.png
|   |— training_curves.png
|   |— metrics_comparison.png
|   |— classification_report.txt
|— docs/
|   |— pseudocode.md
|— LICENSE
```

13.2 README Contents

- Project title
- Problem statement
- Dataset description
- Installation instructions
- Required libraries
- Preprocessing steps
- Training procedure
- Testing procedure
- Model architecture
- Results
- Confusion matrix
- Sample predictions
- How to run the application

14. Assessment Rubric Self-Mapping

Criteria	Marks	Addressed Section
Problem Understanding & CV Fundamentals	15	Section 1
Image Preprocessing, Enhancement & Feature Analysis	20	Sections 4 & 7
Traditional CV vs Deep Learning	20	Section 3
Solution Design & Model Selection	15	Section 4
Analysis, Evaluation & Justification	10	Section 10
Implementation, Results & Visualization	10	Sections 7–9
Reflection, Industrial Relevance & SDG Mapping	10	Section 12
Total	100	All Sections

15. References

1. Li, Z., Wei, X., Hassaballah, M., Li, Y., & Jiang, X. (2023). **A deep learning model for steel surface defect detection.** *Complex & Intelligent Systems*.
2. Gao, Y., Lv, G., Xiao, D., Han, X., Sun, T., et al. (2024). **Research on steel surface defect classification method based on deep learning.** *Scientific Reports*, 14, 8254. \
3. Zhao, B., Chen, Y., Jia, X., & Ma, T. (2024). **Steel surface defect detection algorithm in complex background scenarios.** *Measurement*, 237, 115189.
4. Liu, L.-J., Zhang, Y., & Karimi, H. R. (2024). **Resilient machine learning for steel surface defect detection based on lightweight convolution.** *The International Journal of Advanced Manufacturing Technology*, 134, 4639–4650.
5. Han, J., Cui, G., Li, Z., & Zhao, J. (2024). **DBCW-YOLO: A Modified YOLOv5 for the Detection of Steel Surface Defects.** *Applied Sciences*, 14(11), 4594.
6. Li, H., Liu, M., & Sun, W. (2024). **AFS-YOLO: Steel Surface Defect Detection Algorithm Based on Enhanced Multi-Layer Fusion Network.** *Studies in Applied Electromagnetics and Mechanics*.
7. **Steel surface defect detection and segmentation using deep neural networks.** (2025). *Results in Engineering*.
8. Ge, Y., Li, Z., & Meng, L. (2025). **YOLO-MSD: A robust industrial surface defect detection model via multi-scale feature fusion.** *Applied Intelligence*, 55, 840.
9. **Surface defect detection using distributed features.** (2025). *Engineering Applications of Artificial Intelligence*, 151, 110655.
10. Suh, S. (2025). **Optimal surface defect detector design based on deep learning for 3D geometry.** *Scientific Reports*, 15, 5527.